Understanding Convolutional Neural Networks :Architecture and Applications For Expression Recognition

Name: Laxmi Narayana Gugulothu

Student ID: 24180164

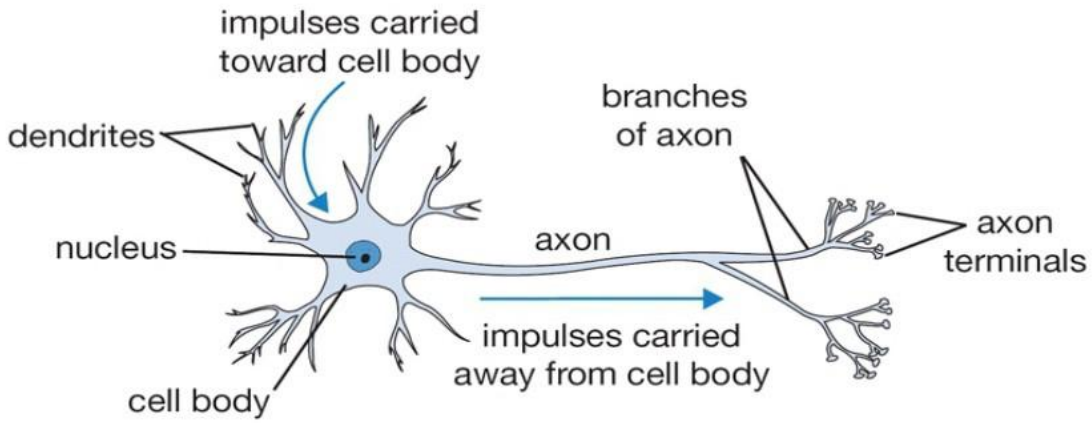
Module: Machine Learning & Neural Network

GitHub Repository: https://github.com/laxminarayana19/CNN-Facial-Expression-Recognition-tutorial.git

Introduction to Convolutional Neural Networks

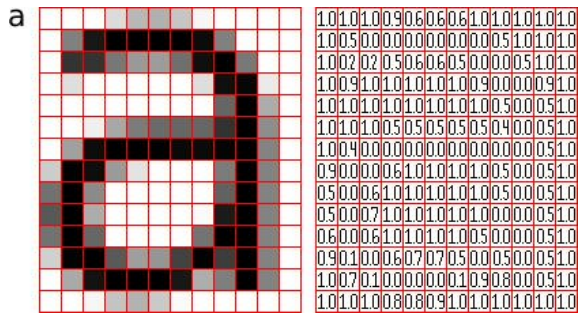
What Is "NN" in "CNN" ?

NN is extensible as Neural Networks which is biologically inspired.



Convolutional Neural Networks (CNNs) are powerful deep learning models used for processing image data. Unlike traditional networks, CNNs keep the spatial structure of images, helping them understand visual patterns more effectively.

Figure 1: Representation of image as a grid of pixels (Source)



They use small filters (kernels) to scan images and detect features like edges, textures, and shapes. These features are learned step by step, from simple patterns in early layers to complex objects in deeper layers. Techniques like parameter sharing and pooling make CNNs efficient and reduce complexity.

Because of these strengths, CNNs are widely used in areas such as image classification, facial recognition, medical imaging, and autonomous driving.

Convolutional Neural Network Architecture

A Convolutional Neural Network (CNN) is made up of several specialized layers that work together to extract features from input data and perform predictions. While CNNs can become very deep and complex, their core architecture is typically built from three main types of layers: convolutional layers, pooling layers, and fully connected layers.

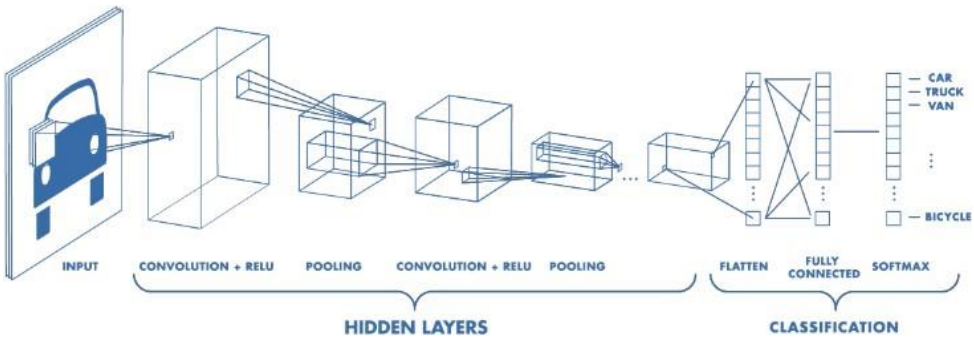


Figure 2: Architecture of a CNN

Convolution Layer

The convolution layer is the core component of a CNN, responsible for extracting important features from input images using learnable filters (kernels). These filters slide across the image and perform a dot product with local regions (receptive fields), producing an output called a **feature map**.

The convolution operation can be expressed as:

$$y(i,j) = \sum_m \sum_n x(i+m, j+n) \cdot w(m,n)$$

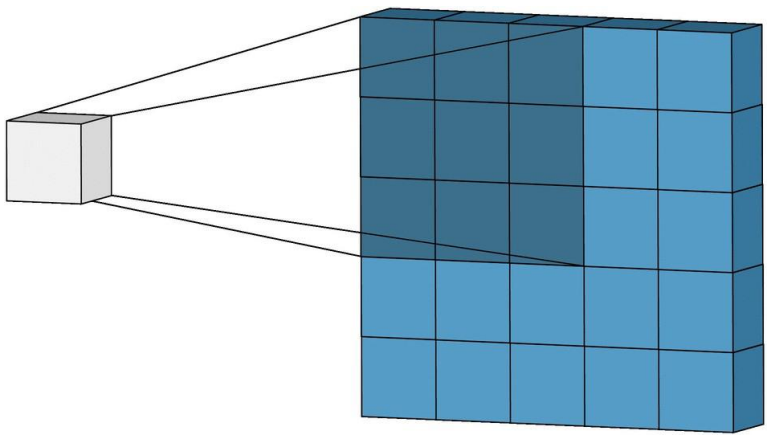


Illustration of Convolution Operation

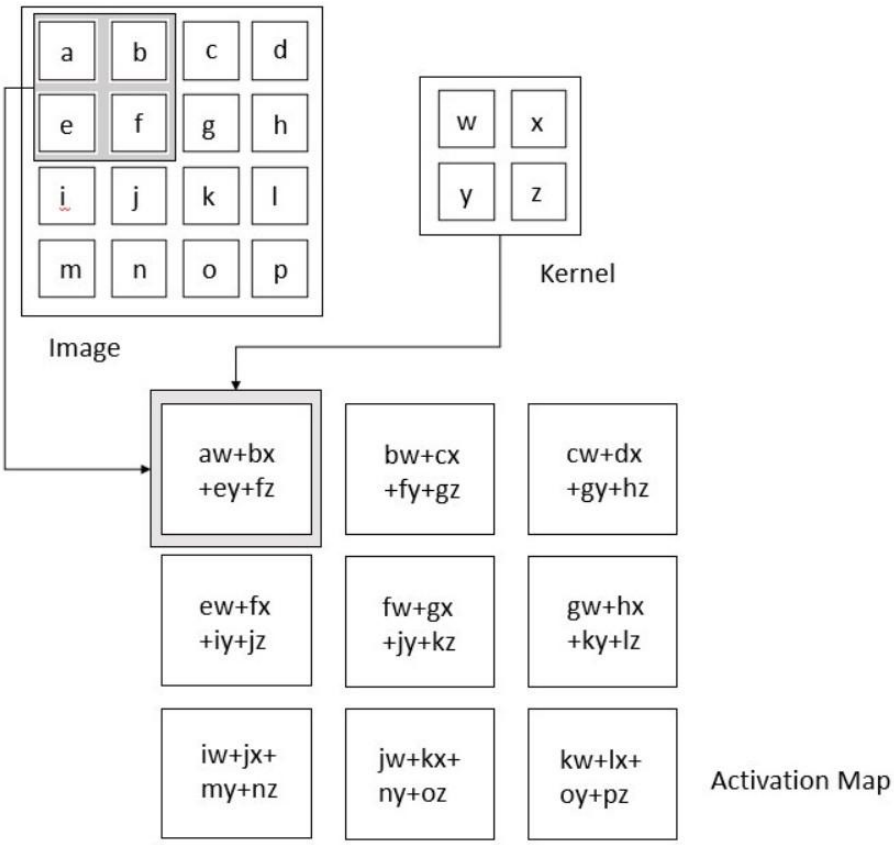
During the forward pass, the kernel scans the image and generates a feature map where higher values indicate strong feature detection and lower values indicate weak detection. The movement of the kernel is controlled by the **stride (S)**—a smaller stride gives more detailed output, while a larger stride reduces size and computation.

The output size is calculated as:

$$\frac{W - F + 2P}{S} + 1$$

Using multiple filters produces an output volume of size:

$$W_{out} \times W_{out} \times D_{out}$$



CNNs are efficient due to three key principles: sparse interaction, parameter sharing, and translation equivariance. These allow the network to detect features efficiently and recognise patterns regardless of their position in the image.

Pooling Layer

Pooling layers in CNNs reduce the spatial size of feature maps while preserving important information. They combine values from small regions into a single value, helping to reduce computation and prevent overfitting.

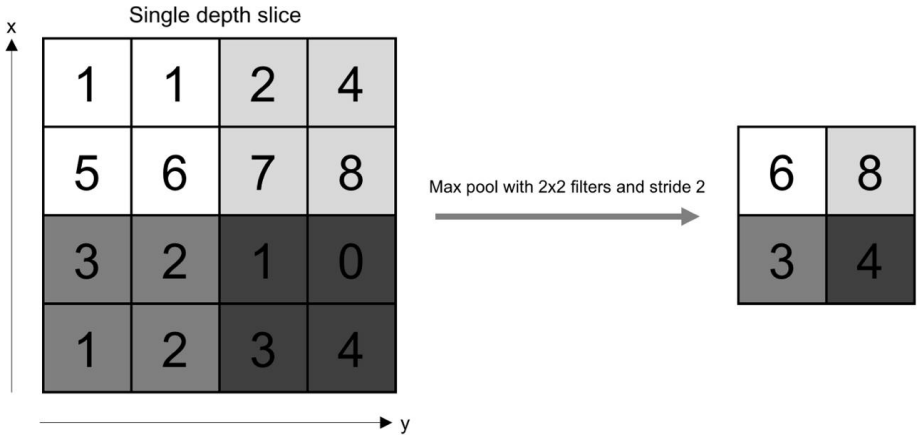


Figure 4: Pooling Operation

Pooling can be local (e.g., 2×2 windows) or global. The most common types are **max pooling**, which selects the highest value, and **average pooling**, which computes the mean. Max pooling is more widely used as it retains the strongest features.

Pooling also provides **translation invariance**, allowing the network to recognise objects even if their position changes.

The output size can be calculated using:

$$\frac{W - F}{S} + 1$$

Fully Connected layer:

A fully connected layer in a CNN links every neuron to all neurons in the previous layer, similar to a multilayer perceptron. Before this stage, feature maps are flattened into a single vector. The layer computes weighted sums, adds biases, and applies activation functions. It combines extracted features to perform high-level reasoning and classification. Essentially, it serves as the decision-making component, transforming learned features into final outputs such as class probabilities for predictions.

Non-Linearity (Activation) Layers

Convolution is a linear operation, which means that stacking multiple convolutional layers without modification would still result in a linear transformation of the input. However, real-world data such as images is complex and non-linear.

To address this, CNNs use **activation functions** after convolutional layers to introduce non-linearity. These functions transform the activation maps, allowing the network to learn complex patterns and relationships in the data. Without activation functions, even a deep network would behave like a simple linear model and would not be able to capture meaningful features.

Sigmoid

The sigmoid activation function is defined as $\sigma(x) = 1/(1+e^{-x})$. I
It converts any real-valued input into a range between 0 and 1, making it useful for probability-based outputs.

However, sigmoid has some limitations. At extreme values, the gradient becomes very small, leading to the **vanishing gradient problem**, which slows down learning during backpropagation. Additionally, its outputs are not zero-centered, which can make training less efficient.

Limitations:

- Suffers from the vanishing gradient problem, where gradients become extremely small at extreme values
- Slows down learning during back propagation
- Outputs are not zero-centered, which can lead to inefficient optimization

---

Tanh (Hyperbolic Tangent)

Tanh squashes a real-valued number to the range [-1, 1]. Like sigmoid, the activation saturates, but — unlike the sigmoid neurons — its output is zero centered.

Limitations:

- Still suffers from vanishing gradients
- Saturates at extreme values, reducing learning efficiency

ReLU (Rectified Linear Unit)

The Rectified Linear Unit (ReLU) has become very popular in the last few years. It computes the function $f(x)=\max(0,x)$. In other words, the activation is simply threshold at zero.

Limitations:

- Can suffer from the "dying ReLU" problem, where neurons stop activating if they consistently receive negative inputs.
- The CNN architecture is structured as follows:

[INPUT]

→ [CONV 1] → [BATCH NORM] → [ReLU] → [POOL 1]

→ [CONV 2] → [BATCH NORM] → [ReLU] → [POOL 2]

→ [FULLY CONNECTED LAYER] →[RESULT]

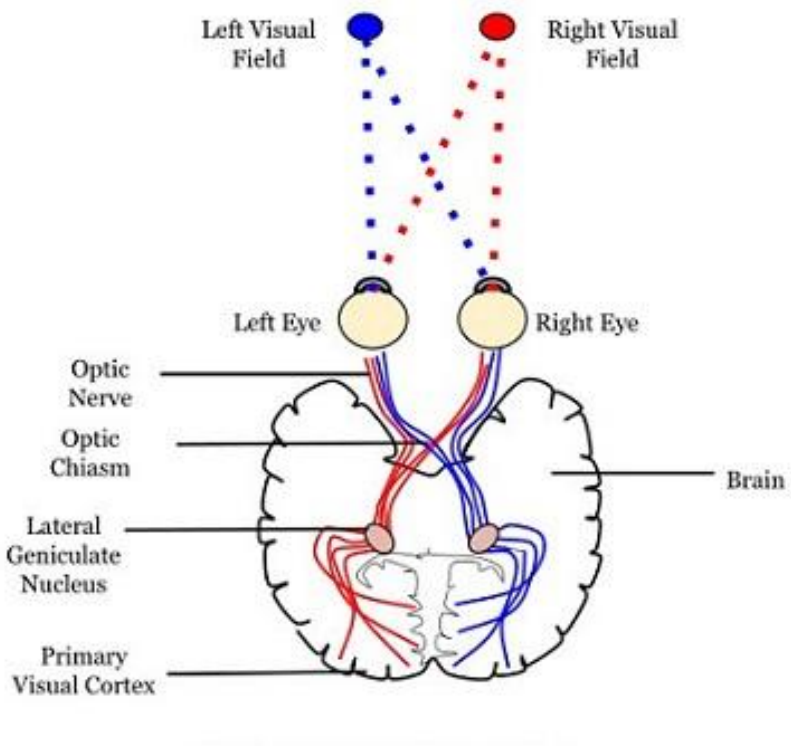
WHY CNN?

- Convolution Neural Networks is a neural network that has some type of specialization which is able to pick patterns and make sense out of it.
- Convolutional Layers have filters which detect the patterns in an image are multiple edges , shapes , textures , objects etc. They are basically good at image classifications
- What is convolutional neural networks?

  CNNs are deep learning models suited for analysing visual imagery. They are heavily influenced by how we - humans, see the surrounding world. Before we proceed to CNNs in order to analyse how computers *see*, let's focus on how humans are able to do it.
- Now, let's see how humans see convolutional neural networks.

How do humans see ?

- When we see an object, the light receptors in your eyes send signals via the optic nerve, where the input is being processed. The primary visual cortex makes sense of what the eye sees.
- Without going into further details, human brain analyses images in the layers of increasing complexity. The first layer distinguishes basic attributes like lines and curves. At higher levels, the brain recognizes that a combination of edges and colours is, for instance, a train or a dog.

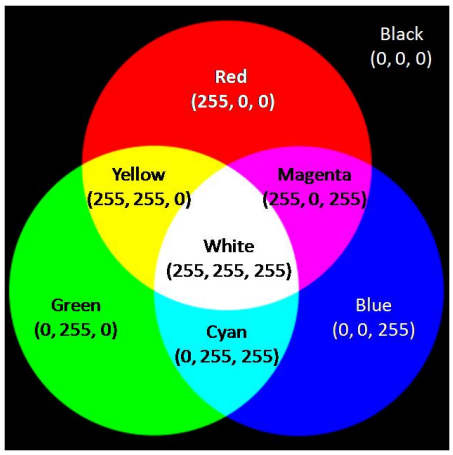


Visual structure of human brain

How do computers see?

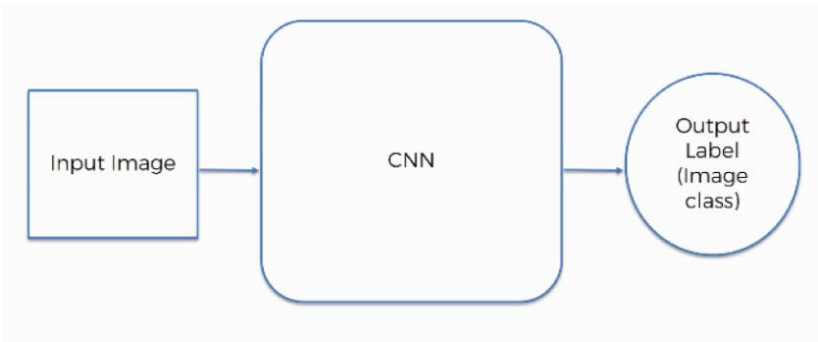
For a computer, an image is simply a grid of numerical values, usually represented as a 3D matrix of RGB pixels. Each pixel contains values for red, green, and blue that define its colour.

CNNs process these images using filters (kernels) that detect specific features such as edges and textures. As the image passes through different layers, the network learns more complex patterns. The ultimate goal of a CNN is to understand and identify what is present in the image.



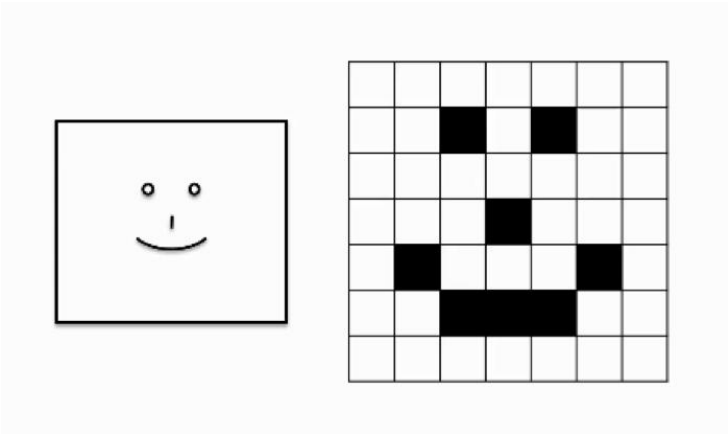
How do CNN actually operate?

- The first thing you need to know here is the elements that are included in the operation:
    - Input image
    - Convolutional Neural Network
    - Output label (image class)
- These elements interact in the following manner:



Detecting facial expressions

In this representation, white cells are shown as 0s and black cells as 1s, meaning the image only

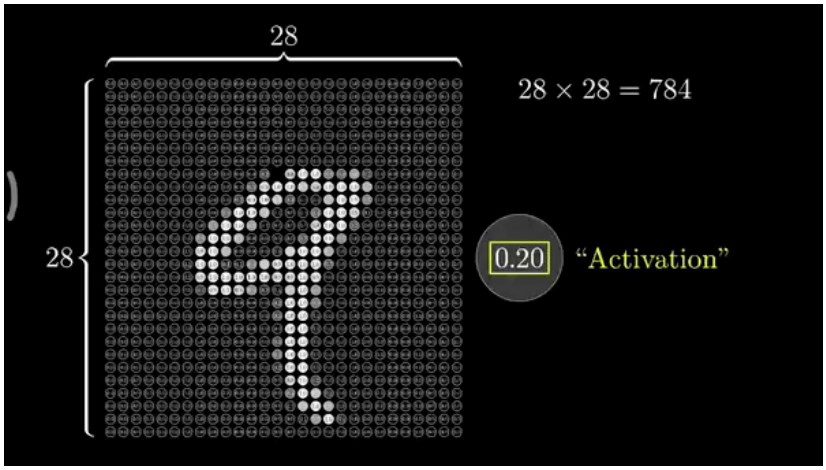


contains two possible values. When training a CNN to detect smiles, the model learns to recognise specific patterns of 0s and 1s that form the shape of a smile.

Steps of convolutional networks

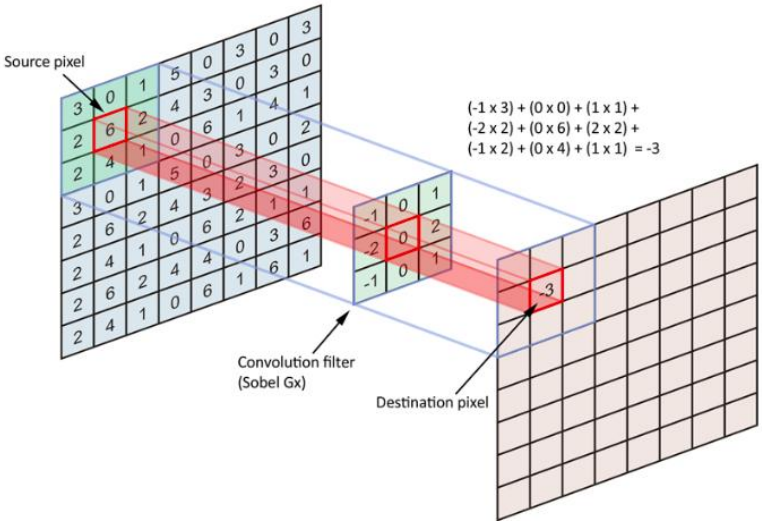
Convolution operation

Now, let's understand how the convolution operation actually works in neural networks. The following example will break down the process step by step in a simple way. To make it more interesting, we'll take a small numerical example and see how different layers in the network process the input and transform it.

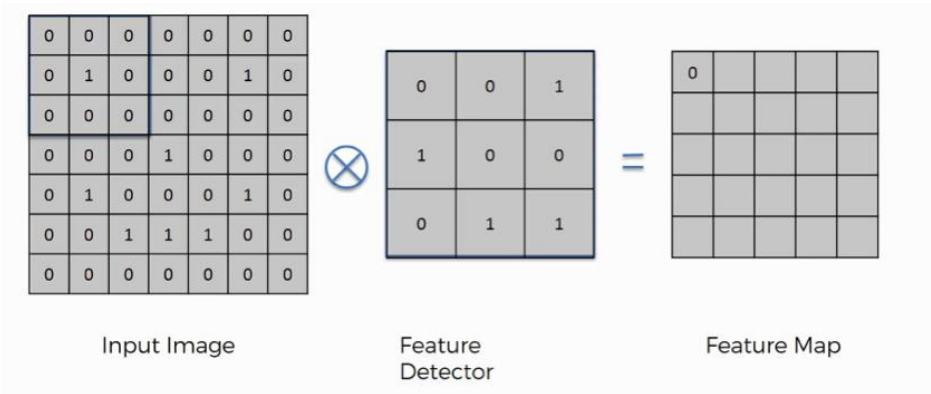


The convolution operation involves three main components: the input image, the feature detector, and the feature map. The input image (like the smiley example) is processed using a small matrix called a feature detector, also known as a kernel or filter. This filter scans the image to identify important patterns, and the result of this process is the feature map, which highlights the detected features.

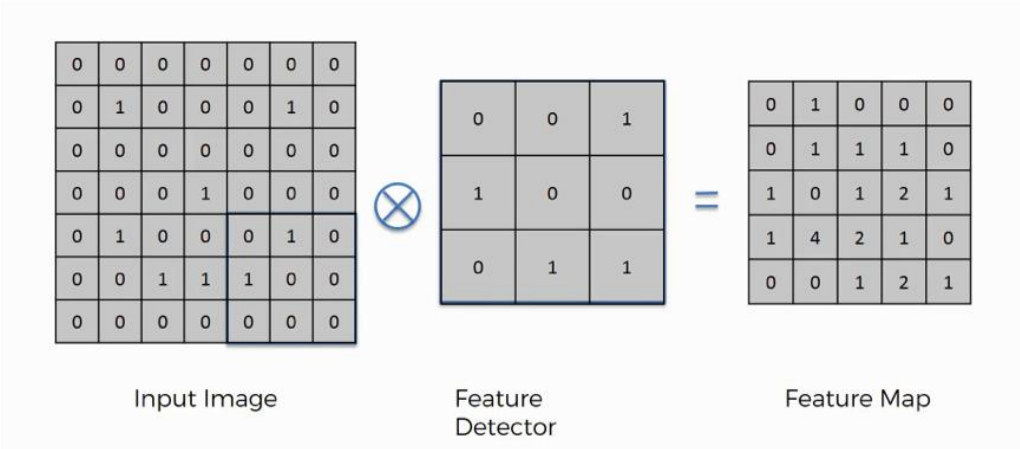
Feature map



- Before going over every pixel

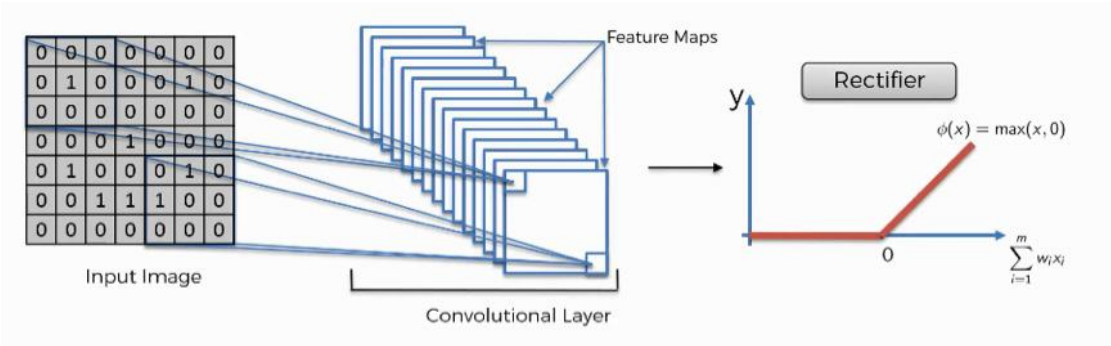


- After going over every pixel



ReLU Layer

ReLU (Rectified Linear Unit) is a widely used activation function in CNNs. It outputs 0 for negative values and keeps positive values unchanged. Its range is from 0 to infinity, and it helps the network learn faster by adding non-linearity.



Max pooling

What is Max Pooling?

Instead of verbally defining pooling, we'll start off this by e with an example right away.

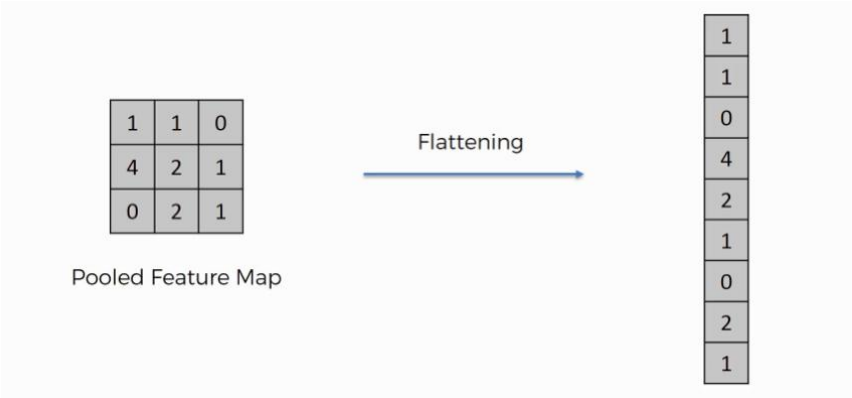
The Cheetah Example:-

Here, we have multiple images of cheetahs taken from different angles, positions, and environments. Even though the images look different, they all represent the same object. Max pooling helps the CNN learn this by focusing on the most important features and ignoring small variations like position, lighting, or background. This allows the network to recognise the cheetah regardless of these differences.

Flattening:

This step is quite simple. After the previous steps, we obtain a pooled feature map, which is then **flattened** into a one-dimensional vector. This means converting the 2D data into a single column of values.

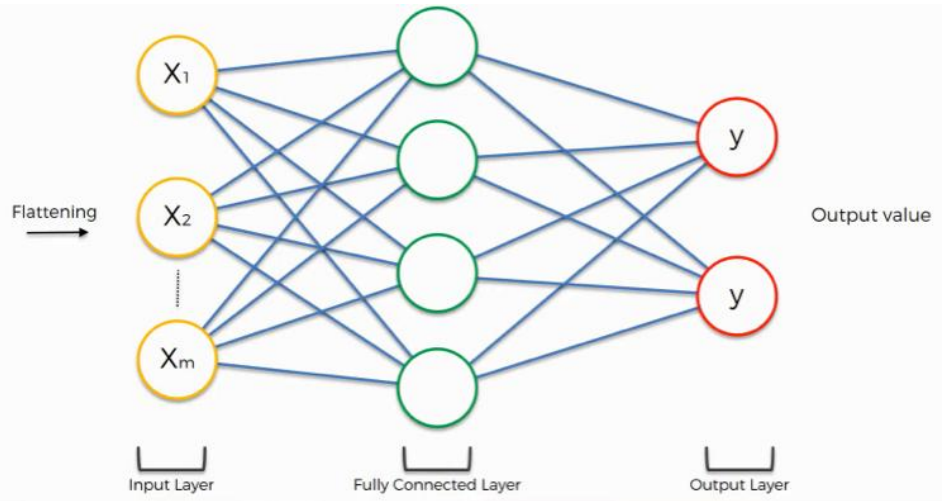
The purpose of flattening is to prepare the data so it can be fed into a traditional neural network. This long vector is then passed to the fully connected layers for further processing and final prediction.



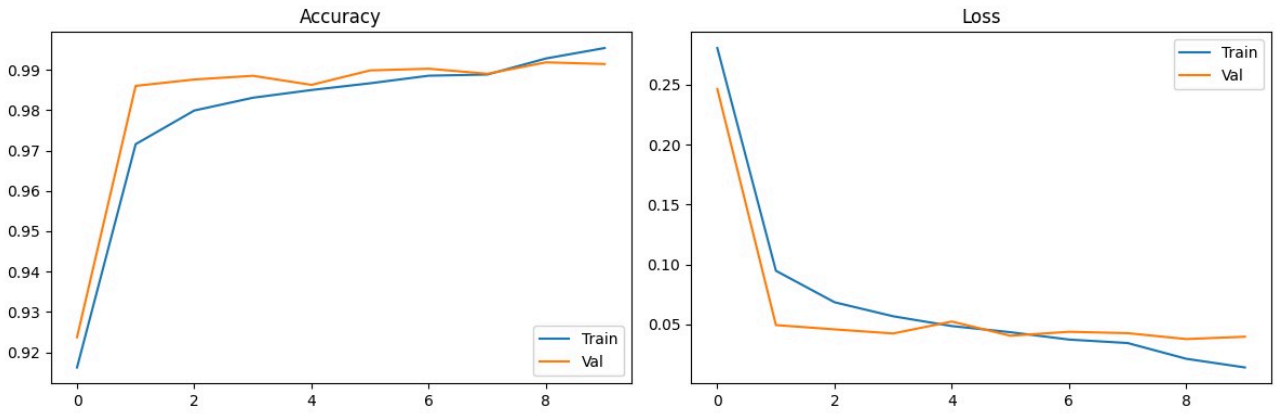
Full connection

This is where CNNs connect with traditional neural networks in the **fully connected stage**. The extracted features are passed through input, fully connected, and output layers to make final predictions.

Each feature is evaluated by different classes (e.g., “dog” or “cat”), and higher weights indicate stronger relevance. By repeating this process during training, the network learns to prioritise important features and make accurate predictions.



Model Training Progress



In this tutorial, the graphs show how the model performs during training. The accuracy graph shows that both training and validation accuracy increase and stay close, meaning the model is learning well and generalizing properly. The loss graph shows that error decreases over time, indicating improved performance.

Accessibility

This tutorial is designed to be accessible to learners with different abilities. It uses clear language and a well-structured format with headings (11 pt body text and 13 pt subheadings) to improve readability and support screen readers. Visual elements are explained with descriptive text, and colour is not solely relied upon, making it suitable for colour-blind users. Code examples are clearly written with comments, and consistent formatting enhances readability. The combination of text, visuals, and practical examples supports different learning styles, ensuring the tutorial is inclusive and accessible.

Conclusion

In conclusion, Convolutional Neural Networks (CNNs) are highly effective models for processing and understanding image data. They use layers like convolution, pooling, and fully connected layers to automatically extract and learn important features from images.

By learning in a hierarchical way, CNNs can identify simple patterns first and then build up to complex structures like objects and facial expressions. Concepts such as parameter sharing and translation equivariance make them efficient and capable of recognising patterns regardless of their position.

Overall, CNNs play a major role in modern AI and are widely used in applications like image classification, facial recognition, and computer vision tasks.

REFERENCES

[1] G. Sumalatha, "Facial Expression Recognition Using CNN," International Journal of Innovative Research in Technology (IJIRT), vol. 8, issue 11, April 2022.

[2] M. El Ayadi, F. Karray, and M. S. Kamel, "Survey on emotion recognition," Pattern Recognition, 2011.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, 2015.

[4] J. Schmidhuber, "Deep learning in neural networks: An overview," Neural Networks, 2015.

[5] R. Godbole and S. Burad, "Face Expression Detection using CNN," International Journal of Innovative Research in Technology (IJIRT), vol. 5, issue 12, May 2019.